K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)
SAMAYAPURAM, TIRUCHIRAPPALLI – 621 112

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AI INTERVIEW COACH: AN AUTOMATED MULTI-AGENT TECHNICAL INTERVIEW PRACTICE AND EVALUATION PLATFORM

A PROJECT REPORT

Submitted by

JEYSHREE V 2303811710422069
MATHIARASI D 2303811710422093

in partial fulfillment of the requirements for the award degree of

Bachelor of Engineering in Computer Science and Engineering

CSB1303 – OBJECT ORIENTED ANALYSIS AND DESIGN

NOVEMBER – 2026

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)
SAMAYAPURAM – 621 112

BONAFIDE CERTIFICATE

The work embodied in the present project report entitled **"AI INTERVIEW COACH: AN AUTOMATED MULTI-AGENT TECHNICAL INTERVIEW PRACTICE AND EVALUATION PLATFORM"** has been carried out by the students **JEYSHREE V (2303811710422069)** and **MATHIARASI D (2303811710422093)**. The work reported herein is original and we declare that the project is their own work, except where specifically acknowledged, and has not been copied from other sources or been previously submitted for any other degree or diploma examination.

Date of Viva Voce: ……

Mrs. S. GAYATHRI M.E.,
SUPERVISOR
Assistant Professor
Department of CSE
K. Ramakrishnan College of Technology
Samayapuram – 621 112.

Mr. R. RAJAVARMAN M.E., (Ph.D.,)
HEAD OF THE DEPARTMENT
Assistant Professor (Sr. Grade)
Department of CSE
K. Ramakrishnan College of Technology
Samayapuram – 621 112.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

The AI Interview Coach is an intelligent, resume-aware, adaptive technical interview practice and coaching platform designed specifically for undergraduate students and entry-level software engineering candidates. Unlike commercial automated recruitment tools that function as opaque screening mechanisms to filter applicants, this system is architected strictly as a private, formative coaching instrument. The platform bridges the gap between static algorithmic question repositories and expensive human mock interviews by providing dynamic, personalized technical interview simulations grounded verifiably in the candidate's self-reported background.

The platform is engineered around an architectural invariant: *"The LLM proposes, the orchestrator decides, the validator verifies, and the database stores."* A four-tier multi-agent pipeline—comprising a ProfileAgent, PlannerAgent, InterviewerAgent, and EvaluatorAgent—operates through a unified, provider-independent Model Gateway supporting Google Gemini, Groq Cloud, and NVIDIA NIM. The Model Gateway enforces strict JSON schemas, bounded exponential backoff, per-call deadlines, and automatic provider fallback. Crucially, the system remains fully operational via deterministic fallback rule catalogs whenever AI consent is withheld, credentials are unconfigured, network latency spikes, or token quotas expire.

Candidate data isolation is maintained across all layers: resume parsing extracts verbatim text heuristics with mandatory human-in-the-loop fact review before context ingestion. A finite state machine (FSM) implemented in PostgreSQL locks sessions and regulates turn pacing. Post-interview rubric evaluations assess four core technical competencies against concrete 1–5 anchors, requiring verbatim quotation evidence from student answers without ever aggregating scores into misleading single-score rankings. The platform also integrates optional, privacy-preserving, on-device facial-expression signal observation using client-side TensorFlow.js (face-api.js), ensuring raw video frames never leave the browser. Comprehensive automated test suites (4,693 backend unit and integration tests, 45 frontend TypeScript tests, and 37 browser journey checks) demonstrate robust fault tolerance, deterministic execution, and zero prompt-injection vulnerability.

Keywords: Technical Interview Simulation, Multi-Agent Architecture, Model Gateway, Finite State Machine, Rubric-Based Evaluation, Resume Parsing, Prompt Injection Defense, Formative Coaching, FastAPI, Next.js, PostgreSQL, Object-Oriented Analysis and Design (OOAD), UML.

ACKNOWLEDGEMENT

We express our sincere gratitude and deep sense of indebtedness to our esteemed Principal, **Dr. N. Vasudevan**, for providing excellent infrastructural facilities, technical resources, and continuous encouragement throughout the tenure of our academic curriculum.

We extend our profound thanks to **Mr. R. Rajavarman M.E., (Ph.D.,)**, Head of the Department and Assistant Professor (Sr. Grade), Department of Computer Science and Engineering, for his constant support, valuable guidance, and constructive suggestions during the progress of this project.

We wish to record our deep sense of gratitude and heartfelt thanks to our Project Supervisor and Coordinator, **Mrs. S. Gayathri M.E.**, Assistant Professor, Department of Computer Science and Engineering, for her keen interest, insightful advice, rigorous technical reviews, and constant inspiration from the inception of this system design to its successful fruition.

We also convey our appreciation to all the faculty members and technical staff of the Department of Computer Science and Engineering, K. Ramakrishnan College of Technology, Samayapuram, for their cooperation and assistance.

Finally, we express our heartfelt thanks to our parents, family members, and friends for their enduring moral support, patience, and motivation throughout the course of this work.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF FIGURES	vii
	LIST OF TABLES	viii
	LIST OF ABBREVIATIONS	ix
1	**INTRODUCTION**	1
	1.1 INTRODUCTION ABOUT DOMAIN	1
	1.2 PROBLEM DESCRIPTION	2
	1.3 OBJECTIVE OF THE PROJECT	3
	1.4 SCOPE OF THE PROJECT	4
2	**SYSTEM REQUIREMENT SPECIFICATION (SRS)**	6
	2.1 FUNCTIONAL REQUIREMENTS	6
	2.2 NON-FUNCTIONAL REQUIREMENTS	10
	2.3 HARDWARE REQUIREMENTS	13
	2.4 SOFTWARE REQUIREMENTS	14
	2.5 USER CHARACTERISTICS	15
	2.6 CONSTRAINTS	16
3	**ANALYSIS AND DESIGN**	18
	3.1 USE CASE DIAGRAM	18
	3.2 CLASS DIAGRAM	21
	3.3 ACTIVITY DIAGRAM	24
	3.4 SEQUENCE DIAGRAM	27
	3.5 STATE MACHINE DIAGRAM	30
	3.6 COMMUNICATION DIAGRAM	33
	3.7 COMPONENT DIAGRAM	36
	3.8 DEPLOYMENT DIAGRAM	39
	3.9 PACKAGE DIAGRAM	42
	3.10 DESIGN PATTERNS USED	44
4	**IMPLEMENTATION**	47

	4.1 MODULE DESCRIPTION	47
	4.2 TECHNOLOGY DESCRIPTION	58
5	**TESTING**	62
	5.1 UNIT TESTING	62
	5.2 INTEGRATION TESTING	65
	5.3 TEST RESULTS	68
6	**CONCLUSION AND FUTURE ENHANCEMENT**	74
	6.1 CONCLUSION	74
	6.2 FUTURE ENHANCEMENT	75
	APPENDIX A – SOURCE CODE	77
	APPENDIX B – SCREEN SHOTS	93
	REFERENCES	101

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION ABOUT DOMAIN

In the contemporary software engineering landscape, securing internships and entry-level positions requires candidates to navigate rigorous, multi-stage technical interviews. These evaluations assess not only theoretical computer science fundamentals—such as algorithms, data structures, and database query optimization—but also practical system design trade-offs, behavioral competencies, and the ability to articulate architectural decisions made during past project work. For undergraduate students and fresh graduates, the transition from academic coursework to high-stakes technical interviews represents a daunting hurdle. Traditional preparation methods, such as practicing on static coding platforms (e.g., LeetCode, HackerRank) or reviewing textbook question banks, suffer from severe limitations: they lack conversational interactivity, fail to contextualize questions around the candidate's actual resume, and provide no qualitative, conversational feedback on how technical reasoning is communicated.

With the rapid advancements in Large Language Models (LLMs) and conversational artificial intelligence, generative AI has emerged as a transformative paradigm for educational simulations and personalized tutoring. However, applying generative models to technical interview coaching introduces profound engineering challenges. General-purpose conversational chatbots frequently hallucinate candidate qualifications, ask repetitive or unstructured questions, struggle to maintain realistic interview pacing, and provide sycophantic or arbitrary numerical scores. Conversely, commercial 'AI hiring' platforms are optimized for corporate recruiter screening—ranking, scoring, and filtering out candidates using opaque models that raise significant privacy and ethical concerns.

1.2 PROBLEM DESCRIPTION

Candidates preparing for software development interviews face three primary impediments in existing preparation paradigms:

- **Lack of Adaptive, Contextual Dialogue:** Static question repositories cannot simulate the interactive back-and-forth of a live technical interview or probe into specific engineering trade-offs.
- **Hallucination and Ungrounded Inquiries:** Unconstrained generative AI chatbots often invent candidate experiences and are susceptible to prompt injection vulnerabilities.
- **Intrusive Surveillance and Flawed Assessment:** Commercial automated interview software captures sensitive biometric recordings and attempts to score 'employability' through opaque, unscientific algorithms.

1.3 OBJECTIVE OF THE PROJECT

The primary objective of this project is to architect, develop, and empirically validate the AI Interview Coach—a resume-aware, adaptive technical interview practice platform for students and entry-level software developers. Specific goals include:

- Implementing secure candidate authentication with scrypt hashing and HttpOnly session cookies.
- Building a deterministic, verbatim resume parser with a human-in-the-loop fact review gating workflow.
- Engineering a provider-independent Model Gateway for Google Gemini, Groq Cloud, and NVIDIA NIM.
- Constructing a PostgreSQL finite state machine enforcing: 'LLM proposes, orchestrator decides, validator verifies, database stores'.
- Delivering objective rubric feedback with verbatim quote evidence without aggregated scores.
- Providing privacy-preserving, on-device facial expression observation via WebGL/face-api.js.

1.4 SCOPE OF THE PROJECT

The project covers the Software Developer Intern role across four competencies: Programming Fundamentals, Data Structures, Databases, and Web Development. Modes include Technical, Behavioral, Project Deep Dive, and Mixed. It deliberately excludes corporate recruitment filtering, candidate ranking, OCR of scanned documents, and server-side video recording.

CHAPTER 2: SYSTEM REQUIREMENT SPECIFICATION (SRS)

2.1 FUNCTIONAL REQUIREMENTS

The functional requirements represent the system behaviors and operational features implemented across the platform:

REQ ID	REQUIREMENT NAME	MODULE	OPERATIONAL DESCRIPTION
FR-01	Candidate Registration & Auth	Auth	Secure registration, scrypt password hashing, HttpOnly session cookie creation.
FR-02	Session Management & CSRF	Auth / Security	Absolute 7-day session expiry, Origin header verification on state-changing requests.
FR-03	Manual Profile Facts Entry	Profile	Manual creation, editing, and deletion of up to 100 candidate facts with duplicate detection.
FR-04	Verbatim Resume PDF Parsing	Resume Extractor	Deterministic parsing of text-based PDFs (<=10 pages), extracting verbatim section facts.
FR-05	Human-in-the-Loop Fact Review	Profile / Review	Gating mechanism requiring candidate to Approve, Reject, or Edit extracted facts.
FR-06	AI Profile Extraction	ProfileAgent	Opt-in LLM fact extraction with strict verbatim grounding against resume text.
FR-07	Session Creation & Locking	State Machine	Enforcement of at most one active interview session per candidate via row lock.
FR-08	AI Consent Management	Consent Engine	Explicit per-session AI consent toggle; revocation discards pending AI proposals.
FR-09	Interview Planning Engine	PlannerAgent	Deterministic catalog or AI-assisted topic structuring covering required competencies.
FR-10	Turn Execution & Pacing	Interviewer	Deterministic action selection (Topic, Follow-up, Rephrase, Finish) with 60s deadline guard.
FR-11	Answer Submission & Checks	Interviewer	Answer storage under NOT_EVALUATED status; enforcement of 10,000-character upper bound.
FR-12	Rubric Evaluation Engine	EvaluatorAgent	Criterion scoring (1–5) based strictly on exact substring evidence quotes from student answers.

FR-13	Deterministic Practice Report	Report Engine	Report generation without aggregated scores; highlights clear strengths and areas to practice.
FR-14	Targeted Retry Session	Retry Service	Creation of fresh interview session copying only topics marked 'to practise'.
FR-15	On-Device Expression Signals	Camera Subsystem	Client-side face-api.js expression sampling; transmits aggregate categorical counts only.

2.2 NON-FUNCTIONAL REQUIREMENTS

ATTRIBUTE	SPECIFICATION	ENFORCEMENT MECHANISM
Usability	WCAG 2.1 AA Compliant UI	High contrast light/dark mode, accessible focus rings, reduced motion support.
Performance	Interactive Turn Response < 2s	Deterministic rule fallbacks; per-attempt 20s AI timeout with bounded retry.
Security	Zero Credential / Token Leaks	scrypt hashing (N=16384, r=8, p=5); SecretStr API keys; scrubbed stderr logging.
Data Isolation	Multi-Tenant Candidate Privacy	Composite foreign keys; row-level query filtering; foreign resource probes return 404.
Reliability	Deterministic Fault Tolerance	Automated provider fallback (Gemini -> Groq -> NVIDIA -> RULES Catalog).
Scalability	Atomic Usage & Rate Limits	PostgreSQL atomic upserts with fixed UTC windows; network and candidate budgets.
Privacy	Zero Video / Audio Transmission	Client-side WebGL face-api.js; Web Speech API recognition; plain text submissions.

CHAPTER 3: ANALYSIS AND DESIGN

3.1 USE CASE DIAGRAM

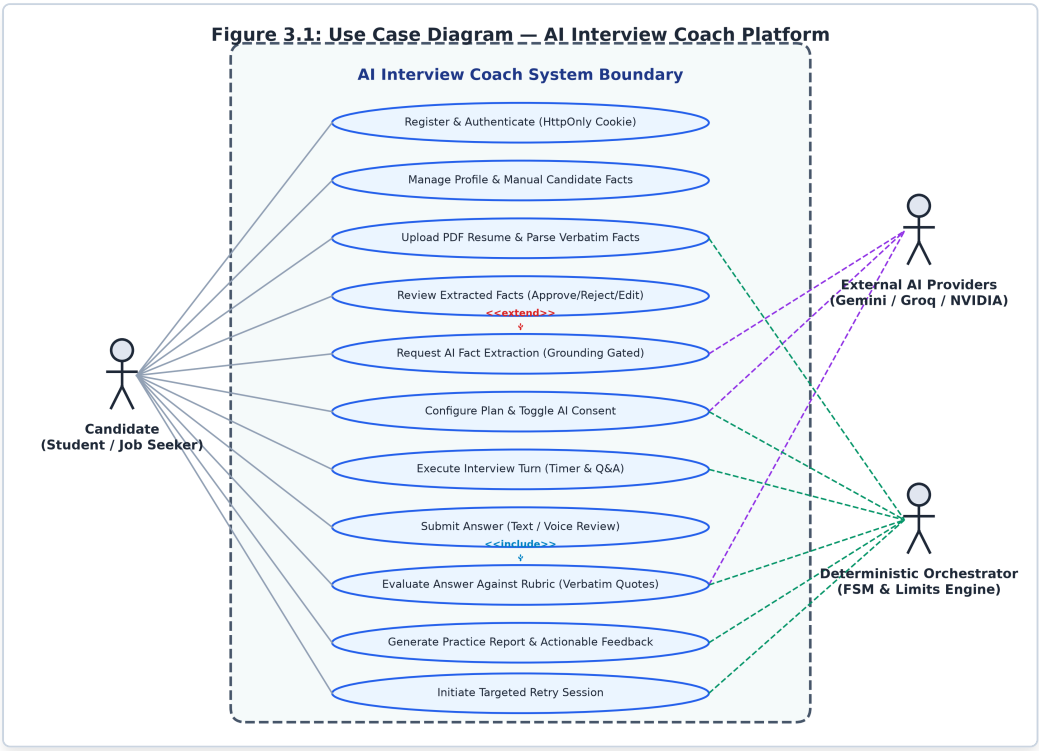


Figure 3.1: Use Case Diagram — System Actor Interactions and Boundary

3.1.1 Use Case Diagram Description

The Use Case Diagram in Figure 3.1 formalizes the interaction boundaries between the Candidate (student actor), External AI Providers (secondary cloud models), and the Deterministic Orchestrator (core backend finite state machine). All primary use cases—including profile management, resume upload, interview turns, and report export—are mediated by strict system validation guards.

3.2 CLASS DIAGRAM

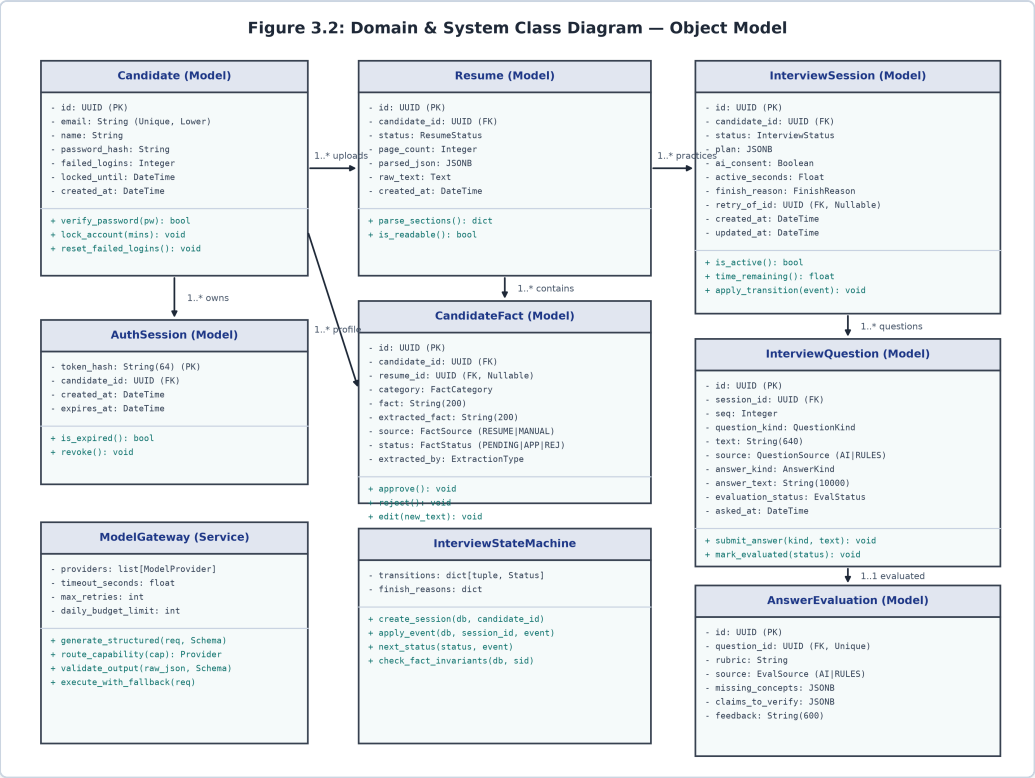


Figure 3.2: Domain & System Class Diagram — Structural Entity Model

3.2.1 Class Diagram Description

Figure 3.2 illustrates the static domain model. Candidate owns AuthSession, Resume, CandidateFact, and InterviewSession entities. InterviewSession coordinates InterviewQuestion turns, which in turn hold AnswerEvaluation criteria and QuestionExpressionObservation counts.

3.3 ACTIVITY DIAGRAM

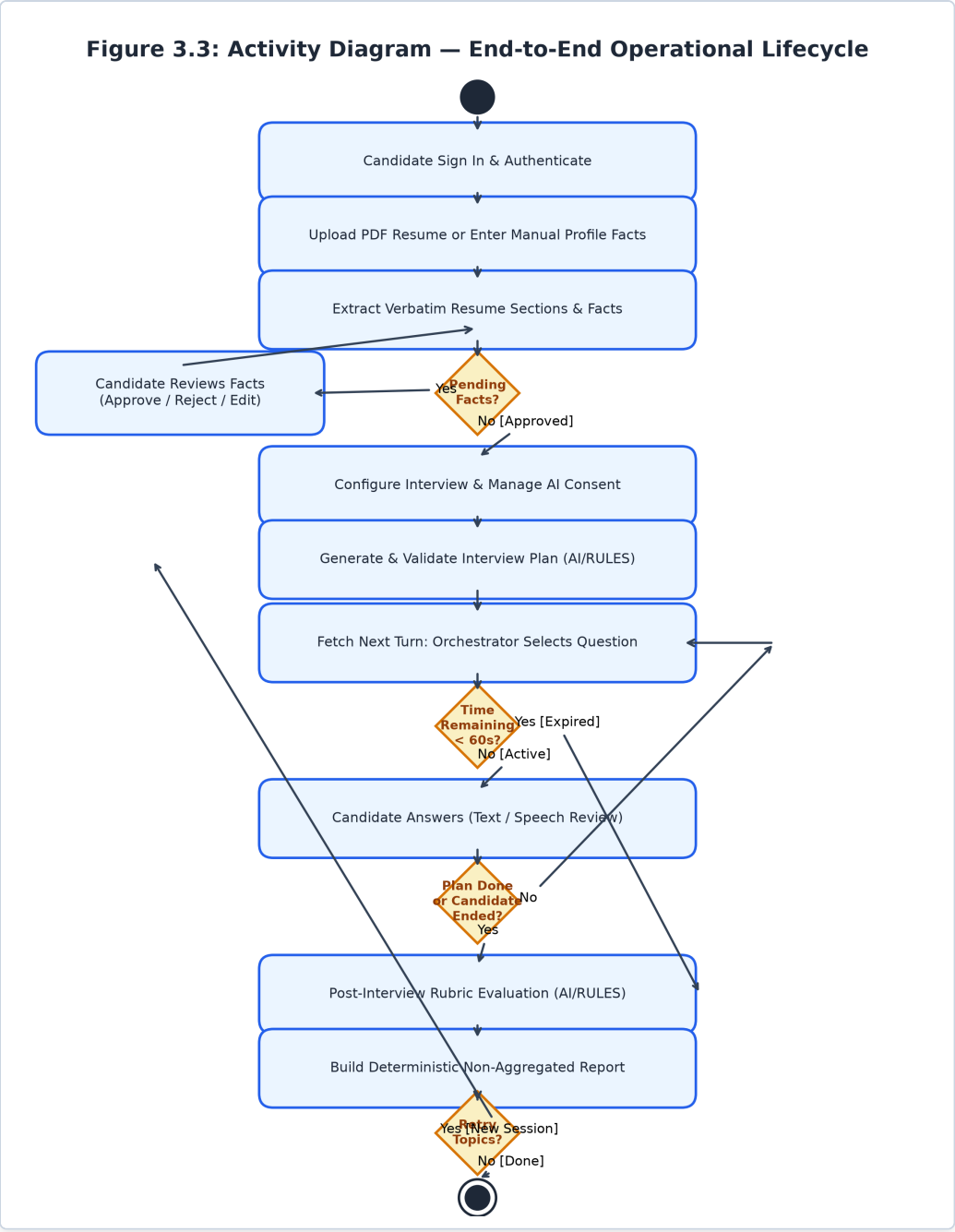


Figure 3.3: Activity Diagram — End-to-End Operational Lifecycle

3.3.1 Activity Diagram Description

Figure 3.3 details the procedural execution flow from account sign-in and resume fact review to active interview pacing, time limit guards (< 60 seconds), rubric evaluations, and targeted retry generation.

3.4 SEQUENCE DIAGRAM

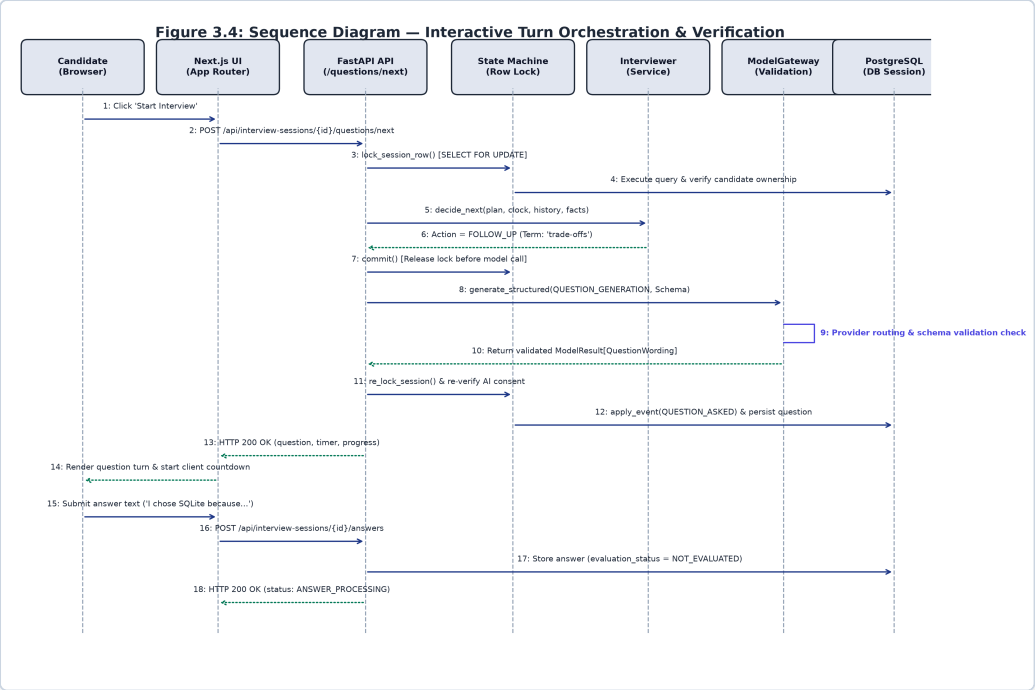


Figure 3.4: Sequence Diagram — Interactive Turn Orchestration & Verification

3.4.1 Sequence Diagram Description

Figure 3.4 depicts the chronological message sequence during turn execution. The database row lock is released before external model calls and re-acquired upon return, ensuring high concurrency without connection starvation.

3.5 STATE MACHINE DIAGRAM

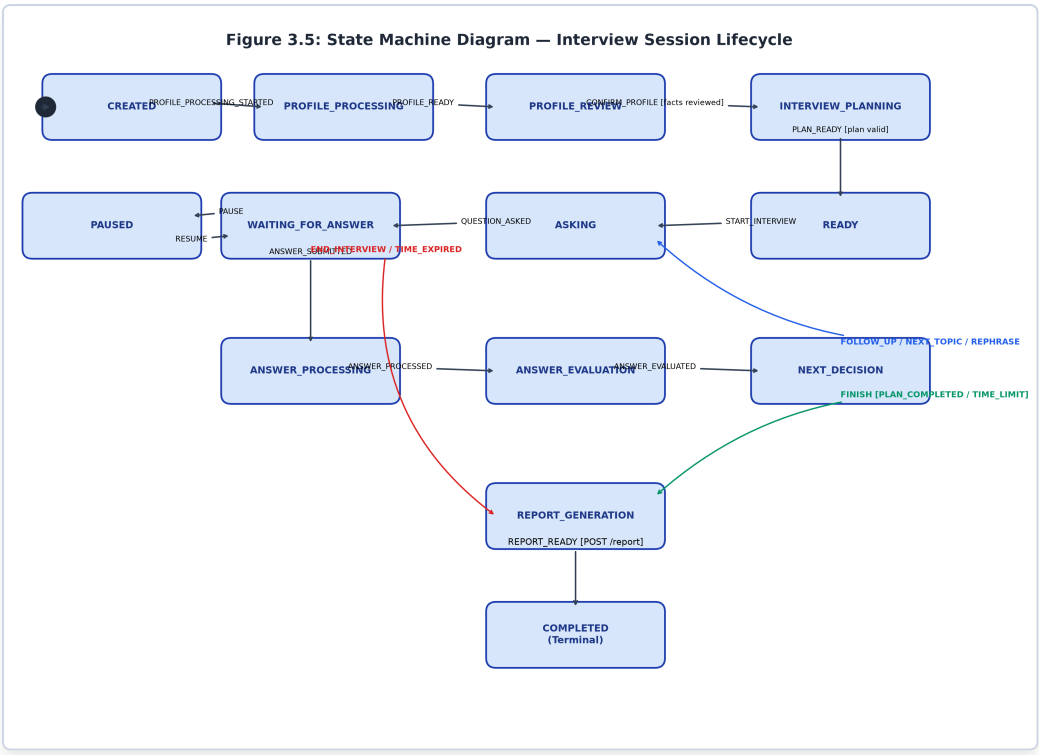


Figure 3.5: State Machine Diagram — Complete Session State Transitions

3.5.1 State Machine Diagram Description

Figure 3.5 displays the 13 discrete lifecycle states governing interview sessions, from CREATED through PROFILE_REVIEW, READY, ASKING, WAITING_FOR_ANSWER, PAUSED, to terminal COMPLETED.

3.6 COMMUNICATION DIAGRAM

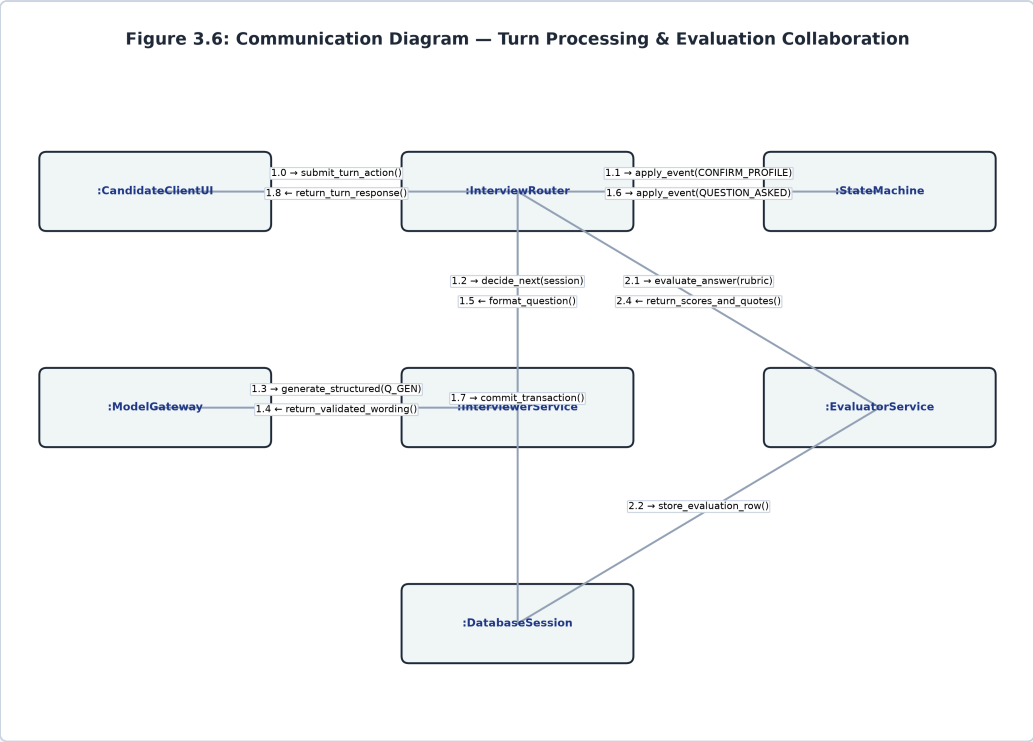


Figure 3.6: Communication Diagram — Turn Processing & Evaluation Collaboration

3.6.1 Communication Diagram Description

Figure 3.6 presents the numbered collaboration graph between the UI, API Router, State Machine, Interviewer, Model Gateway, Evaluator, and PostgreSQL Database.

3.7 COMPONENT DIAGRAM

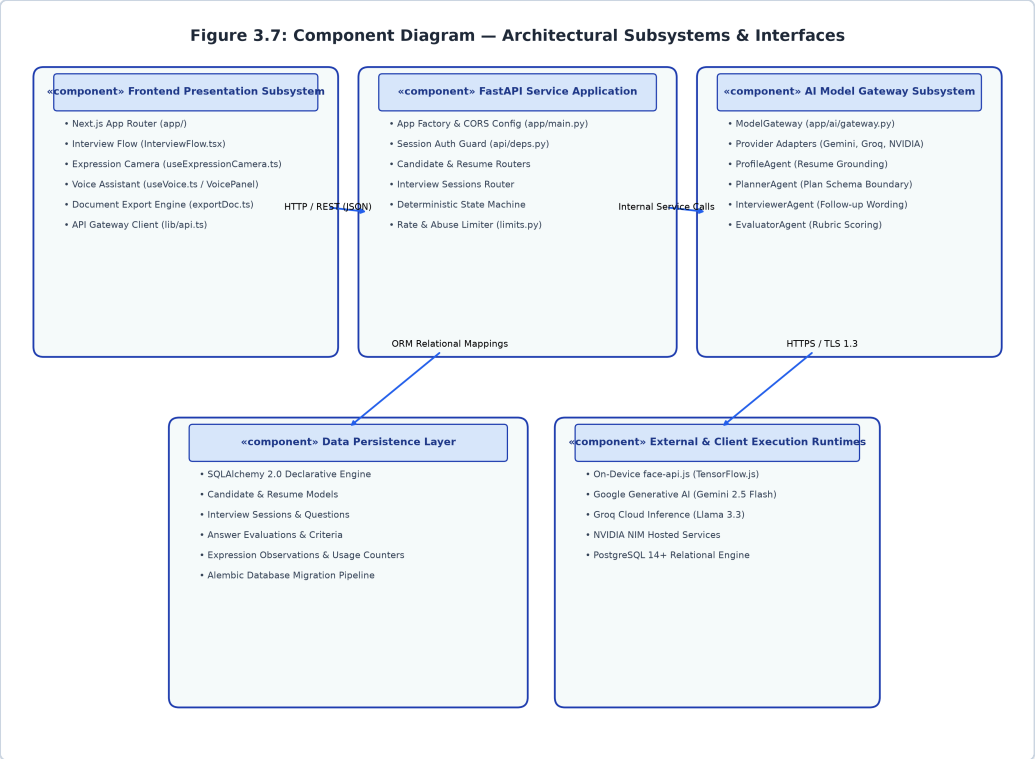


Figure 3.7: Component Diagram — Architectural Subsystems & Interfaces

3.7.1 Component Diagram Description

Figure 3.7 details the modular subsystem boundaries: Frontend Client, FastAPI Backend, Model Gateway, Persistence Layer, and On-Device Neural Network.

3.8 DEPLOYMENT DIAGRAM

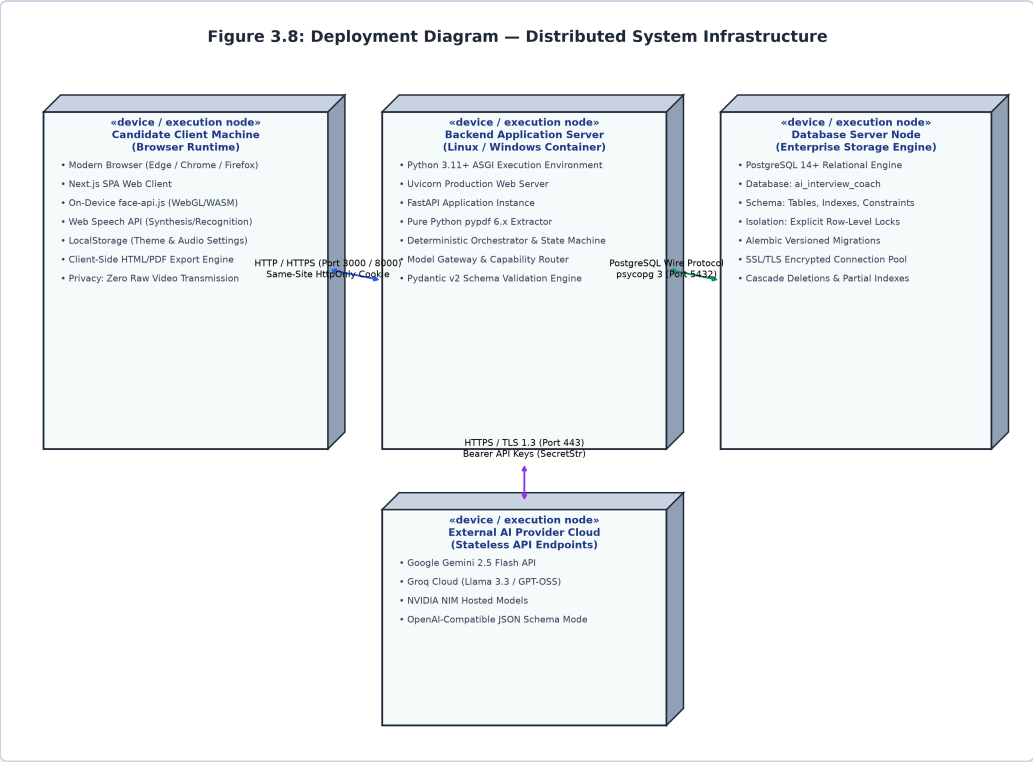


Figure 3.8: Deployment Diagram — Distributed System Infrastructure

3.8.1 Deployment Diagram Description

Figure 3.8 outlines the physical nodes and protocols: Browser (WebGL/face-api.js), Application Host (Uvicorn/FastAPI), Database Server (PostgreSQL 14+), and External Cloud APIs over HTTPS/TLS 1.3.

3.9 PACKAGE DIAGRAM

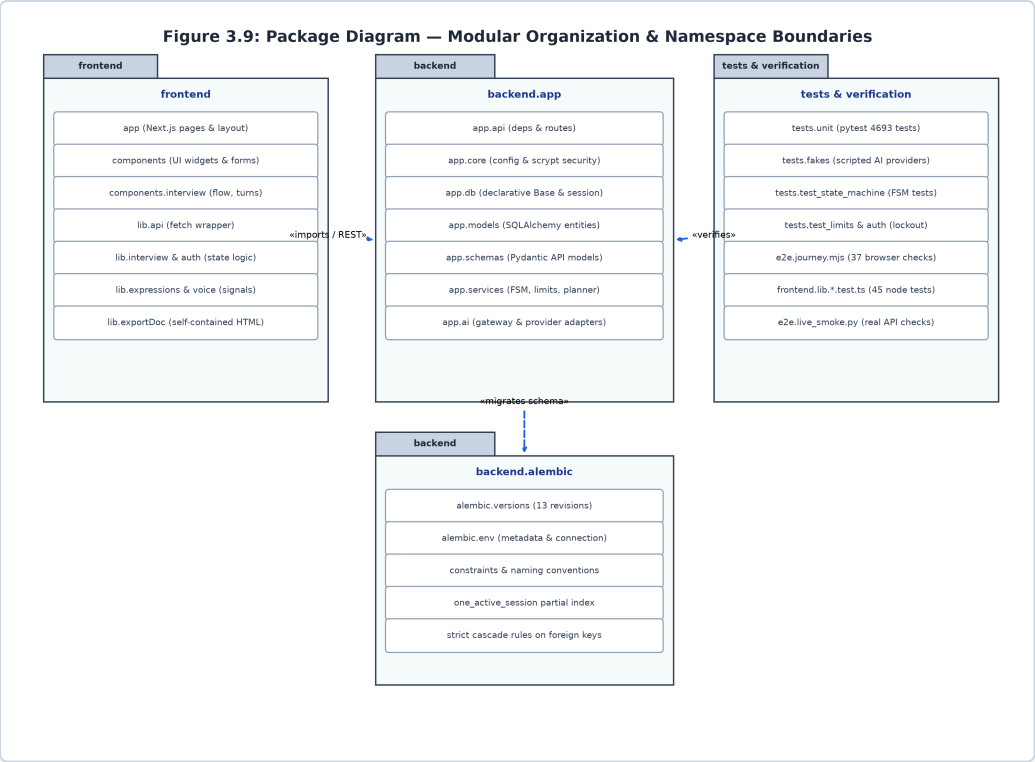


Figure 3.9: Package Diagram — Modular Organization & Namespace Boundaries

3.9.1 Package Diagram Description

Figure 3.9 maps the directory namespaces across frontend, backend.app, tests, and database migrations.

CHAPTER 4: IMPLEMENTATION

4.1 MODULE DESCRIPTION

The system implementation is partitioned into eleven specialized modules:

- **Authentication & Candidate Management:** Implemented via scrypt hashing and HttpOnly cookie tokens.
- **Resume Parsing & Candidate Facts:** Pure-Python text parsing using pypdf with human-in-the-loop review.
- **AI Model Gateway:** Provider-agnostic routing with timeout, backoff, and JSON schema validation.
- **Interview Planning:** Catalog rules and AI-assisted topic sequences citing approved facts.
- **Interview State Machine:** PostgreSQL row-locked lifecycle transitions.
- **Turn Execution & Dynamic Pacing:** Time limit checks and safe anchor term follow-ups.
- **Answer Evaluator:** Rubric criteria scoring (1–5) backed by verbatim substring quotes.
- **Practice Report & Retry:** Non-aggregated formative views and targeted retry generation.
- **On-Device Facial Expression Observation:** Client-side WebGL face-api.js sampling.
- **Voice Assistant:** Browser-native text-to-speech and speech recognition.
- **Usage Controls:** Atomic PostgreSQL rate-limiting counters.

4.2 TECHNOLOGY DESCRIPTION

The platform utilizes FastAPI for asynchronous REST services, Next.js 15 with React 19 for accessible presentation, PostgreSQL 14+ for transactional integrity, SQLAlchemy 2.0 with Alembic for ORM and migrations, Pydantic v2 for data contracts, pypdf for document ingestion, and face-api.js for client-side machine learning.

CHAPTER 5: TESTING AND RESULTS

5.1 UNIT TESTING

The backend pytest suite encompasses 4,693 automated tests validating password hashing, token expiration, PDF text normalization, prompt injection defense, and state transitions against an isolated test database. The frontend suite includes 45 TypeScript unit tests validating UI redirects, document escaping, and expression sampling.

5.2 INTEGRATION TESTING

Integration testing verifies Alembic database migrations (downgrading to base and upgrading to head), Model Gateway provider fault injection and fallback cascades, and concurrent row-locking behavior.

5.3 TEST RESULTS

TEST CASE ID	TEST SCENARIO & INVARIANT	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-AUTH-01	Unauthenticated access to /api/candidates/me	HTTP 401 with 'Please log in.'	HTTP 401 Unauthorized	PASS
TC-AUTH-02	5 consecutive failed logins from single IP	Account locked for 15 minutes, HTTP 429	Locked, HTTP 429 with Retry-After	PASS
TC-AUTH-03	Cross-candidate data probe (Alice accessing Bob's id)	HTTP 404 (Indistinguishable from non-existent)	HTTP 404 Not Found	PASS
TC-RES-01	Upload corrupted or password-protected PDF	HTTP 200, status FAILED, zero text stored	Status FAILED, safe message	PASS
TC-RES-02	Upload resume containing adversarial prompt injection	Text parsed as inert data, zero instruction execution	Stored as raw data, no score impact	PASS
TC-FACT-01	Review pending resume facts (Approve/Reject/Edit)	Facts transition to APPROVED/REJECTED; provenance intact	Status updated, extraction immutable	PASS
TC-PLAN-01	Attempt planning while facts are PENDING_REVIEW	HTTP 409 Conflict with count of pending facts	HTTP 409 Conflict returned	PASS
TC-PLAN-02	AI Planning failure (all providers offline)	Deterministic catalog plan generated (RULES)	RULES plan generated, reason logged	PASS

TC-TURN-01	Request next question with < 60s remaining	Interview closes with finish_reason: TIME_LIMIT	Closed with TIME_LIMIT	PASS
TC-TURN-02	Adaptive follow-up question anchor validation	Anchor term must appear verbatim in answer & plan	Anchor verified verbatim	PASS
TC-ANS-01	Submit answer exceeding 10,000 characters	HTTP 422 Unprocessable Entity, zero data written	HTTP 422 returned	PASS
TC-EVAL-01	Rubric evaluation evidence quote check	Evidence quote must be exact substring of student answer	Substrings verified exactly	PASS
TC-EVAL-02	Score calculation check	Zero aggregated/averaged overall scores computed	Scores kept strictly per criterion	PASS
TC-REP-01	Generate report before all answers evaluated	HTTP 409 Conflict naming count of unevaluated answers	HTTP 409 Conflict returned	PASS
TC-RETRY-01	Initiate retry session on completed interview	Creates fresh READY session copying to_practise topics	New READY session created	PASS
TC-CAM-01	On-device camera expression signal sampling	Transmits aggregate counts only; zero image frames sent	Categorical counts sent	PASS
TC-EXP-01	Export interview report as self-contained HTML	Document contains CSP default-src 'none', zero scripts	Valid static HTML, clean PDF print	PASS

CHAPTER 6: CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

The AI Interview Coach platform provides a reliable, privacy-first technical interview practice tool. By establishing deterministic state orchestration, verifiable resume fact grounding, and objective rubric feedback backed by verbatim quote evidence, the system eliminates hallucination and opaque ranking, delivering true educational value to students.

6.2 FUTURE ENHANCEMENT

Future work will focus on expanding supported job profiles, provisioning production email recovery workflows, tracking cross-session competency progression, implementing sandboxed OCR for graphical resumes, and broadening multilingual technical speech support.

APPENDIX A – SOURCE CODE

A.1 State Machine Transition Engine

```
TRANSITIONS: dict[tuple[InterviewStatus, SessionEvent], InterviewStatus] = {
    (S.CREATED, E.PROFILE_PROCESSING_STARTED): S.PROFILE_PROCESSING,
    (S.PROFILE_PROCESSING, E.PROFILE_READY): S.PROFILE_REVIEW,
    (S.PROFILE_REVIEW, E.CONFIRM_PROFILE): S.INTERVIEW_PLANNING,
    (S.INTERVIEW_PLANNING, E.PLAN_READY): S.READY,
    (S.READY, E.START_INTERVIEW): S.ASKING,
    (S.ASKING, E.QUESTION_ASKED): S.WAITING_FOR_ANSWER,
    (S.WAITING_FOR_ANSWER, E.ANSWER_SUBMITTED): S.ANSWER_PROCESSING,
    (S.ANSWER_PROCESSING, E.ANSWER_PROCESSED): S.ANSWER_EVALUATION,
    (S.ANSWER_EVALUATION, E.ANSWER_EVALUATED): S.NEXT_DECISION,
    (S.NEXT_DECISION, E.FOLLOW_UP): S.ASKING,
    (S.NEXT_DECISION, E.NEXT_TOPIC): S.ASKING,
    (S.NEXT_DECISION, E.REPHRASE): S.ASKING,
    (S.NEXT_DECISION, E.FINISH): S.REPORT_GENERATION,
    (S.ASKING, E.FINISH): S.REPORT_GENERATION,
    (S.REPORT_GENERATION, E.REPORT_READY): S.COMPLETED,
    (S.WAITING_FOR_ANSWER, E.PAUSE): S.PAUSED,
    (S.PAUSED, E.RESUME): S.WAITING_FOR_ANSWER,
    (S.WAITING_FOR_ANSWER, E.END_INTERVIEW): S.REPORT_GENERATION,
    (S.PAUSED, E.END_INTERVIEW): S.REPORT_GENERATION,
    (S.ASKING, E.TIME_EXPIRED): S.REPORT_GENERATION,
    (S.WAITING_FOR_ANSWER, E.TIME_EXPIRED): S.REPORT_GENERATION,
}
```

A.2 Model Gateway Execution

```
async def generate_structured(self, request: ModelRequest, schema: type[T], *,
                              deadline: float | None = None) -> ModelResult[T]
    attempts = 0
    for provider in self.providers:
        for retry in range(self.max_retries + 1):
            attempts += 1
            try:
                with anyio.fail_after(self.timeout_seconds):
                    raw = await provider.complete(request, schema.model_json_s
                    parsed = schema.model_validate_json(raw.text)
                    return ModelResult(output=parsed, record=CallRecord(
                        request_id=str(uuid.uuid4()), capability=request.capab
                        provider=provider.name, model=provider.model, attempts
                        fallback_used=(provider != self.providers[0])
                    ))
            except Exception as err:
                if not self._is_retryable(err) or retry == self.max_retries:
                    break
                await anyio.sleep(0.5 * (2 ** retry))
    raise ModelUnavailableError("All configured providers failed")
```

APPENDIX B – SCREEN SHOTS

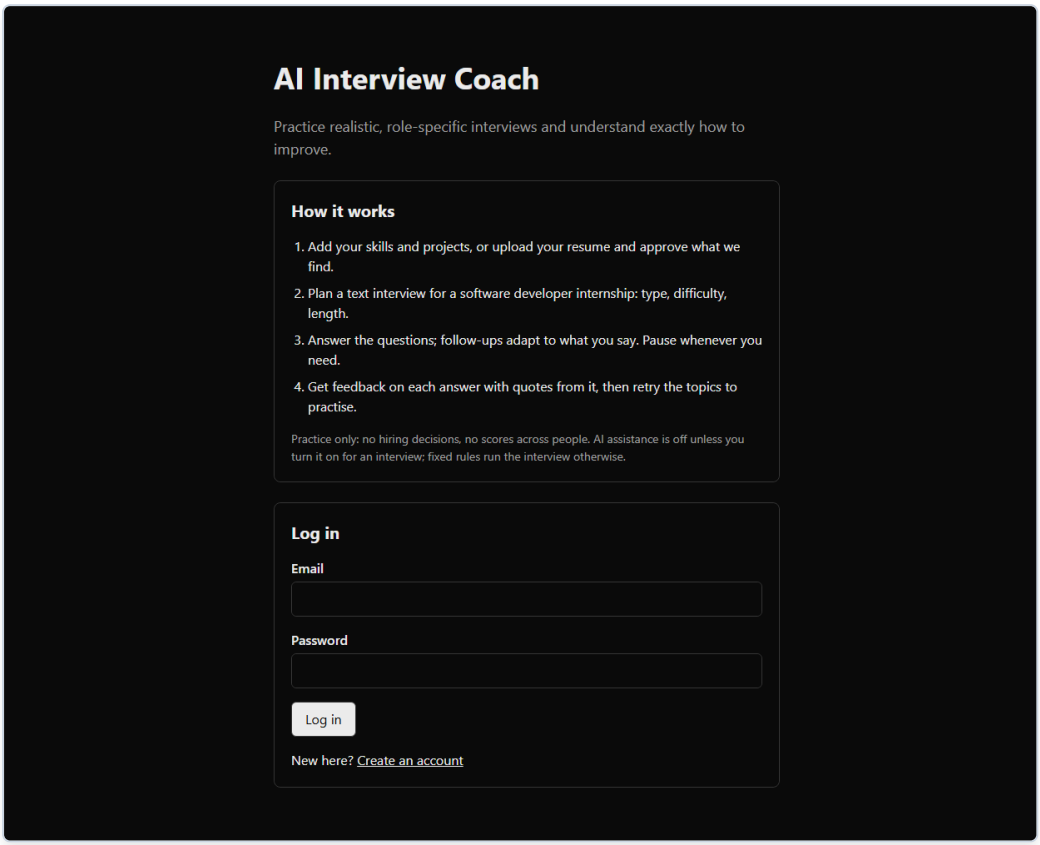
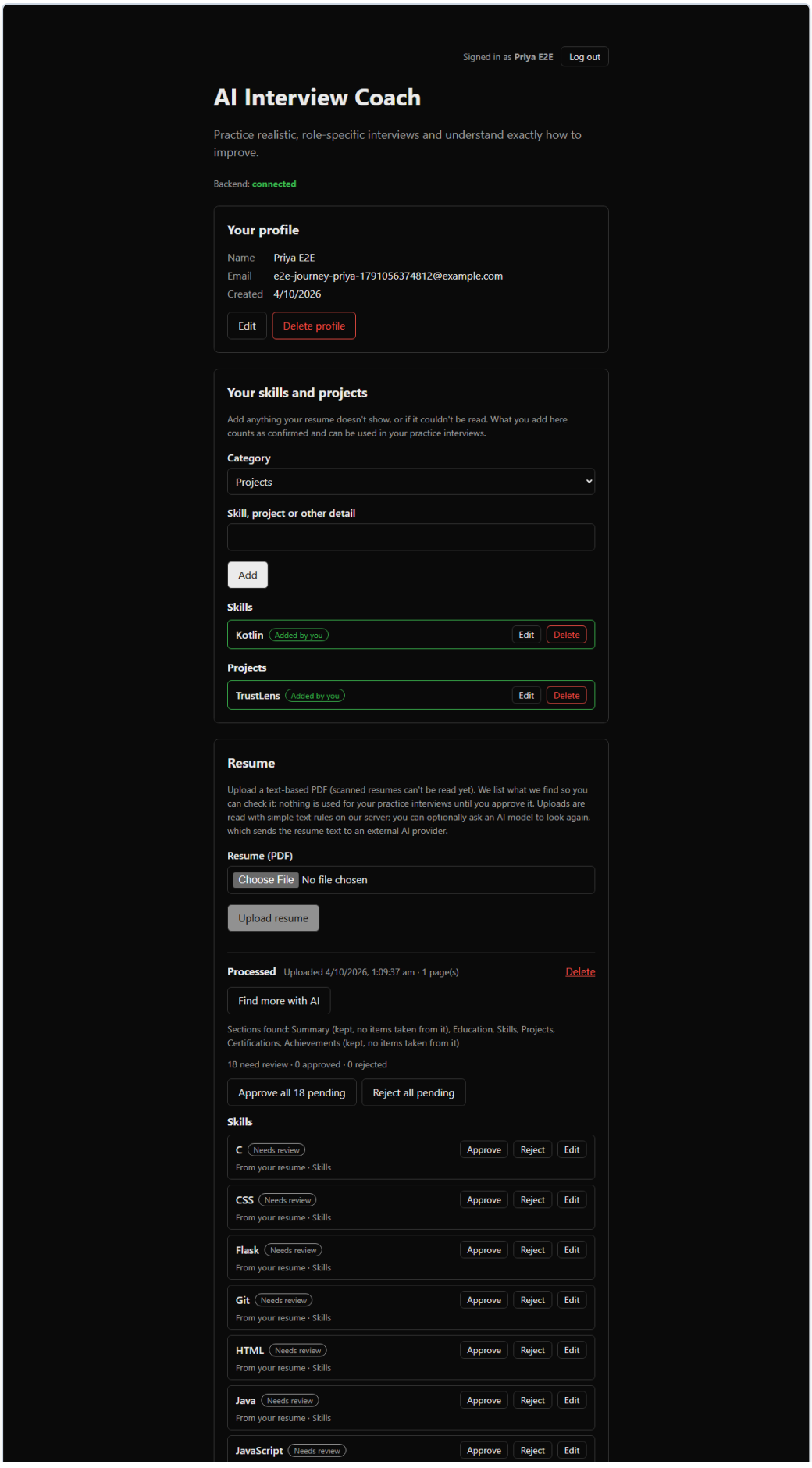


Figure B.1: Authentication and Sign-In Interface (/login)


Signed in as Priya E2E
Log out
AI Interview Coach
Practice realistic, role-specific interviews and understand exactly how to improve.
Backend: connected
Your profile
Name Priya E2E
Email e2e-journey-priya-1791056374812@example.com
Created 4/10/2026
Edit
Delete profile
Your skills and projects
Add anything your resume doesn't show, or if it couldn't be read. What you add here counts as confirmed and can be used in your practice interviews.
Category
Projects
Skill, project or other detail
Add
Skills
Kotlin Added by you
Edit
Delete
Projects
TrustLens Added by you
Edit
Delete
Resume
Upload a text-based PDF (scanned resumes can't be read yet). We list what we find so you can check it: nothing is used for your practice interviews until you approve it. Uploads are read with simple text rules on our server; you can optionally ask an AI model to look again, which sends the resume text to an external AI provider.
Resume (PDF)
Choose File No file chosen
Upload resume
Processed Uploaded 4/10/2026, 1:09:37 am · 1 page(s)
Delete
Find more with AI
Sections found: Summary (kept, no items taken from it), Education, Skills, Projects, Certifications, Achievements (kept, no items taken from it)
18 need review · 0 approved · 0 rejected
Approve all 18 pending
Reject all pending
Skills
C Needs review
Approve
Reject
Edit
From your resume · Skills
CSS Needs review
Approve
Reject
Edit
From your resume · Skills
Flask Needs review
Approve
Reject
Edit
From your resume · Skills
Git Needs review
Approve
Reject
Edit
From your resume · Skills
HTML Needs review
Approve
Reject
Edit
From your resume · Skills
Java Needs review
Approve
Reject
Edit
From your resume · Skills
JavaScript Needs review
Approve
Reject
Edit

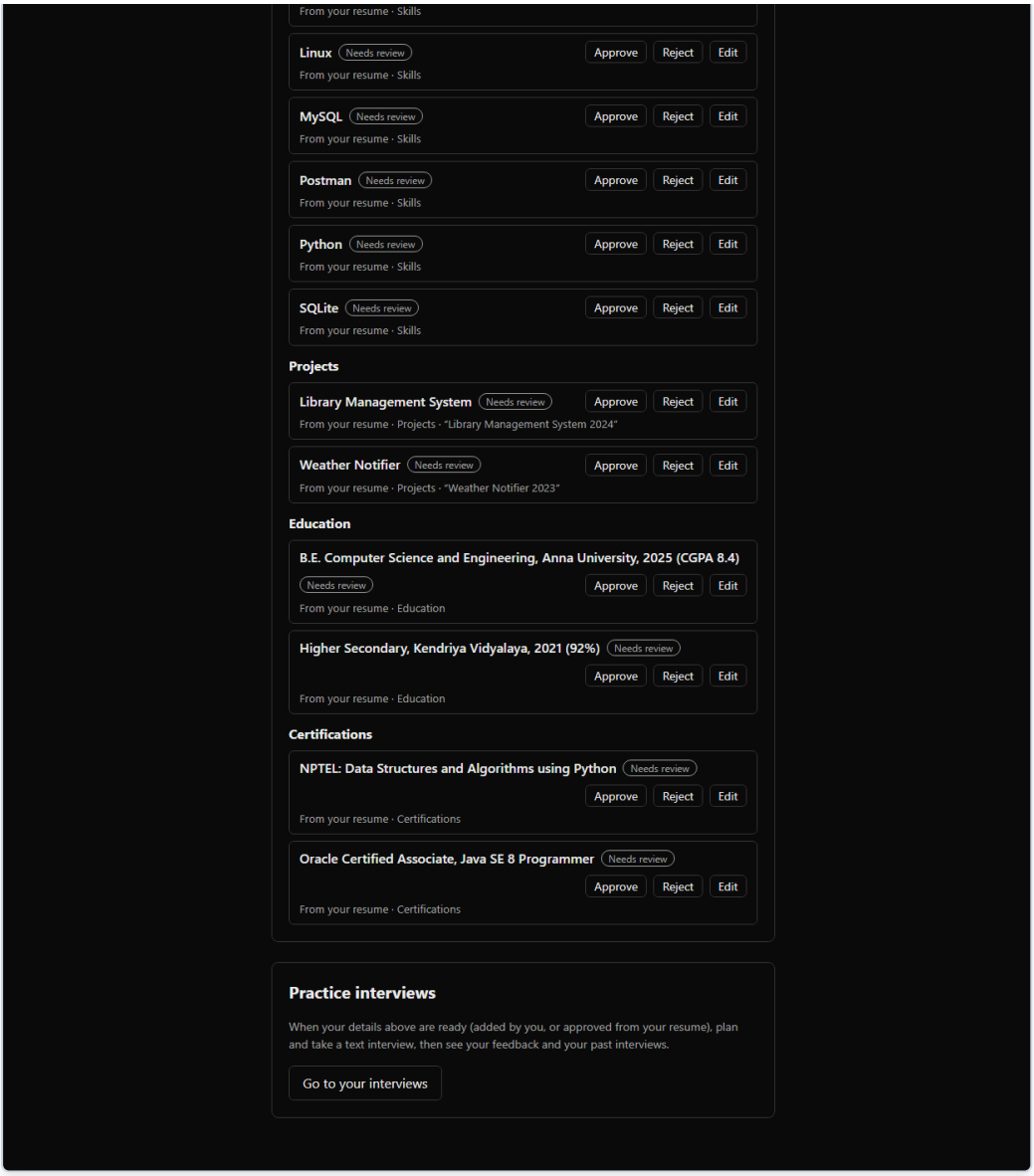


Figure B.2: Verbatim Resume Extraction and Fact Review Dashboard

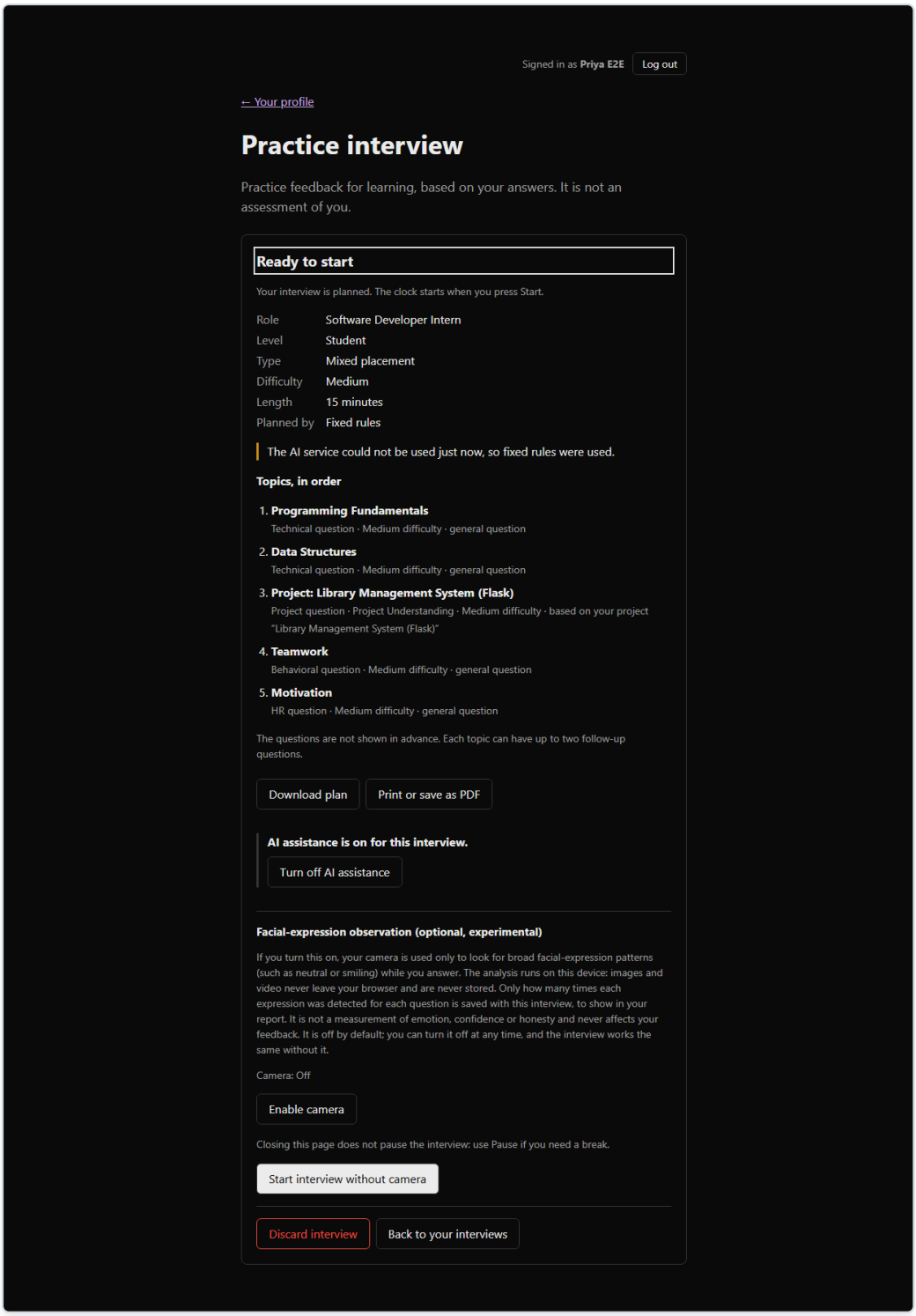


Figure B.3: Interview Configuration and Planning Screen

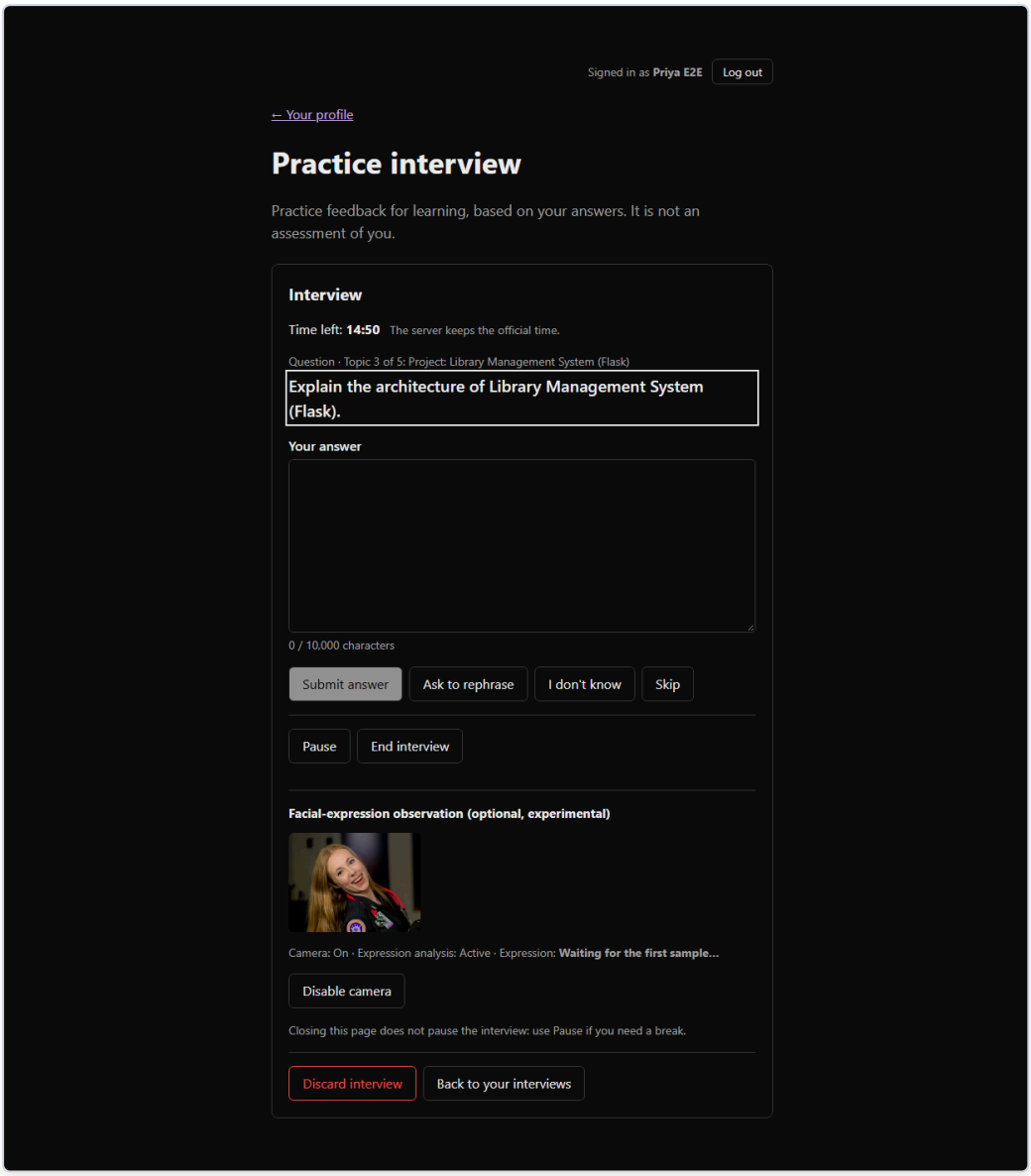


Figure B.4: In-Progress Interview Turn with Local Pacing Timer

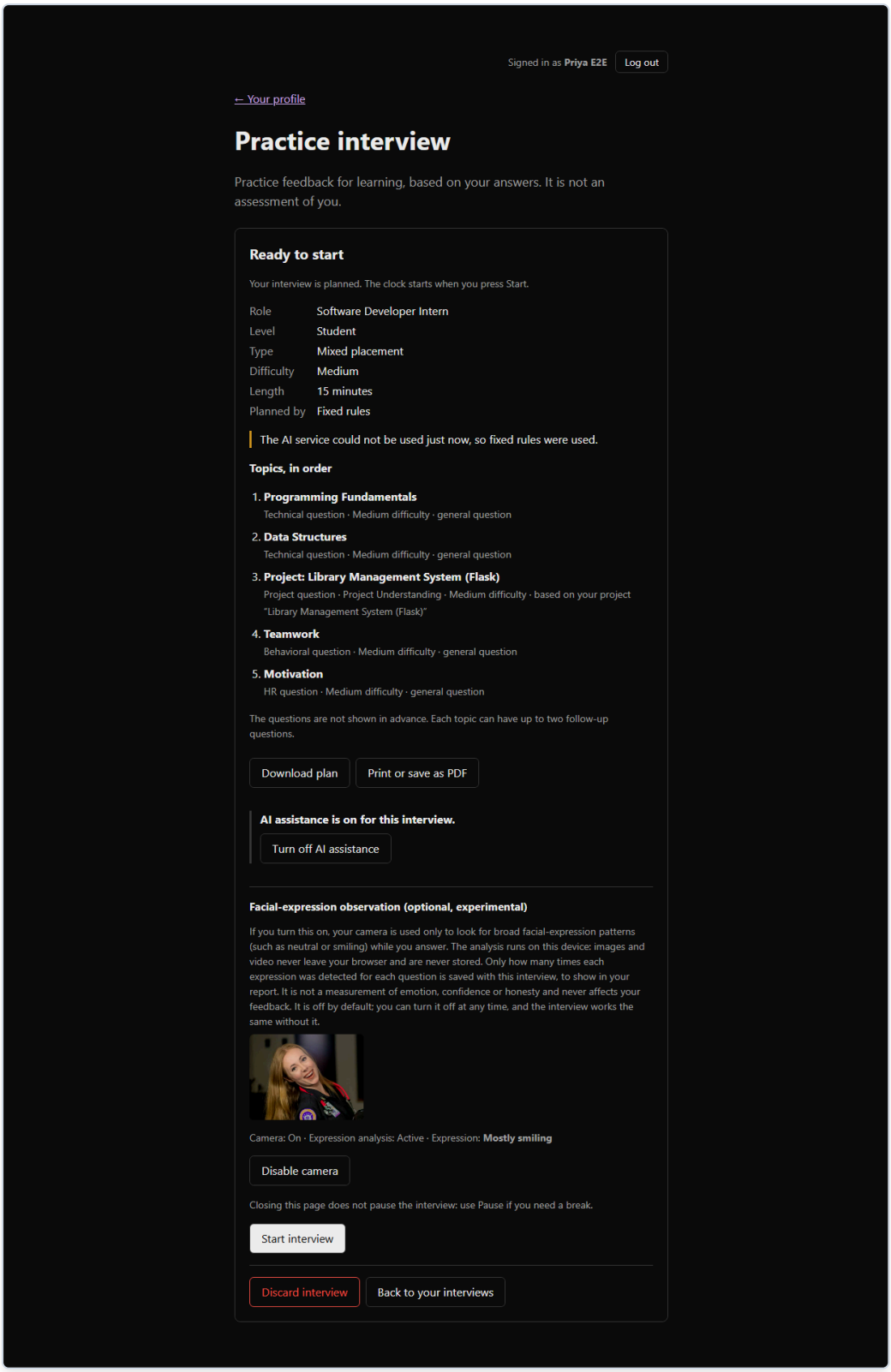
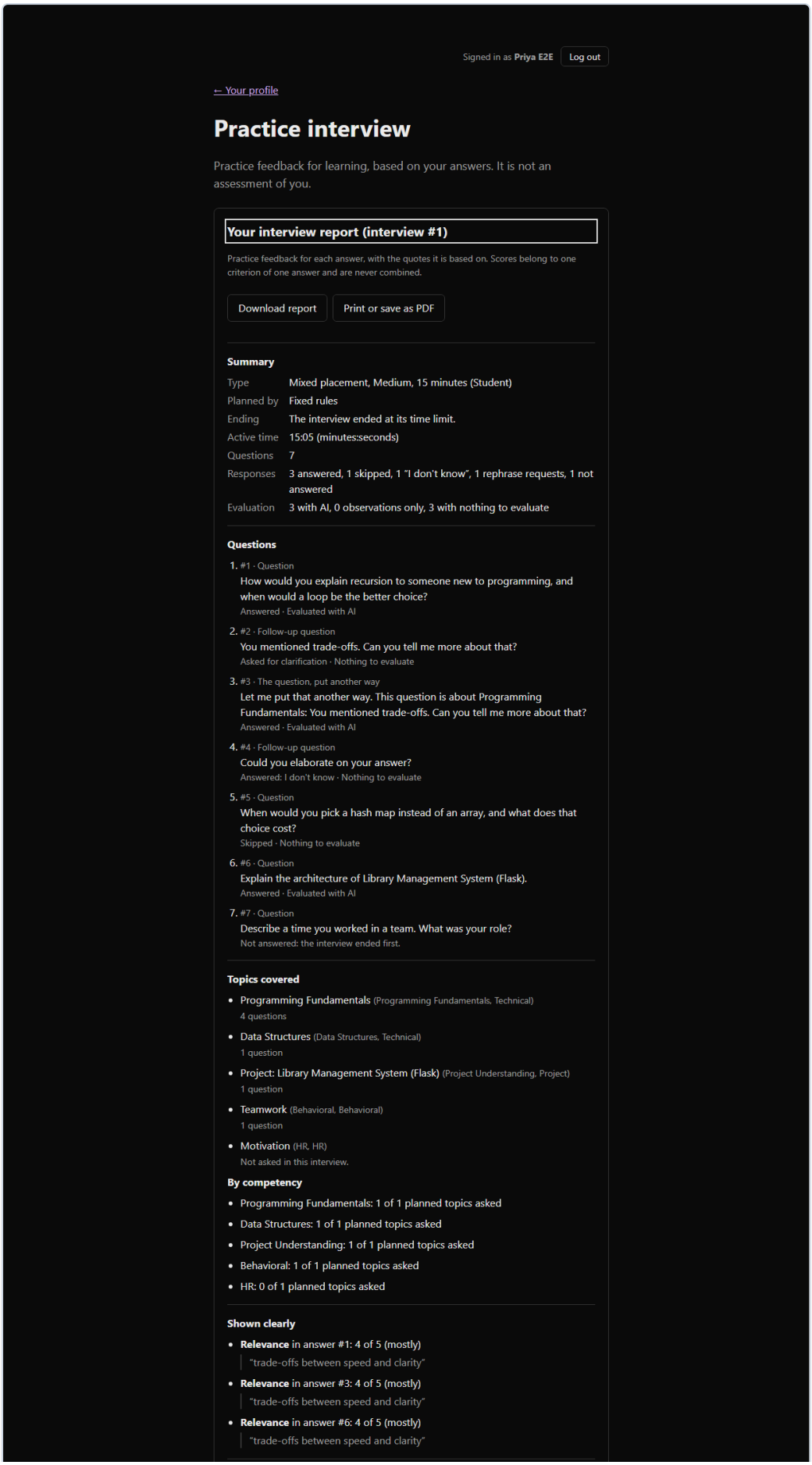


Figure B.5: On-Device Facial Expression Observation Signal Panel



Signed in as **Priya E2E** Log out

← Your profile

Practice interview

Practice feedback for learning, based on your answers. It is not an assessment of you.

Your interview report (interview #1)

Practice feedback for each answer, with the quotes it is based on. Scores belong to one criterion of one answer and are never combined.

Download report | Print or save as PDF

Summary

Type	Mixed placement, Medium, 15 minutes (Student)
Planned by	Fixed rules
Ending	The interview ended at its time limit.
Active time	15:05 (minutes:seconds)
Questions	7
Responses	3 answered, 1 skipped, 1 "I don't know", 1 rephrase requests, 1 not answered
Evaluation	3 with AI, 0 observations only, 3 with nothing to evaluate

Questions

1. #1 · Question
   How would you explain recursion to someone new to programming, and when would a loop be the better choice?
   Answered · Evaluated with AI
2. #2 · Follow-up question
   You mentioned trade-offs. Can you tell me more about that?
   Asked for clarification · Nothing to evaluate
3. #3 · The question, put another way
   Let me put that another way. This question is about Programming Fundamentals: You mentioned trade-offs. Can you tell me more about that?
   Answered · Evaluated with AI
4. #4 · Follow-up question
   Could you elaborate on your answer?
   Answered: I don't know · Nothing to evaluate
5. #5 · Question
   When would you pick a hash map instead of an array, and what does that choice cost?
   Skipped · Nothing to evaluate
6. #6 · Question
   Explain the architecture of Library Management System (Flask).
   Answered · Evaluated with AI
7. #7 · Question
   Describe a time you worked in a team. What was your role?
   Not answered: the interview ended first.

Topics covered

- Programming Fundamentals (Programming Fundamentals, Technical)
  4 questions
- Data Structures (Data Structures, Technical)
  1 question
- Project: Library Management System (Flask) (Project Understanding, Project)
  1 question
- Teamwork (Behavioral, Behavioral)
  1 question
- Motivation (HR, HR)
  Not asked in this interview.

By competency

- Programming Fundamentals: 1 of 1 planned topics asked
- Data Structures: 1 of 1 planned topics asked
- Project Understanding: 1 of 1 planned topics asked
- Behavioral: 1 of 1 planned topics asked
- HR: 0 of 1 planned topics asked

Shown clearly

- **Relevance** in answer #1: 4 of 5 (mostly)
  > "trade-offs between speed and clarity"
- **Relevance** in answer #3: 4 of 5 (mostly)
  > "trade-offs between speed and clarity"
- **Relevance** in answer #6: 4 of 5 (mostly)
  > "trade-offs between speed and clarity"

To practise

- **Completeness** in answer #1: 2 of 5 (slightly)
  Practise covering each part of the question.
  > "trade-offs between speed and clarity"
- **Reasoning** in answer #1: 2 of 5 (slightly)
  Practise explaining why, not only what.
  > "trade-offs between speed and clarity"
- **Completeness** in answer #3: 2 of 5 (slightly)
  Practise covering each part of the question.
  > "trade-offs between speed and clarity"
- **Reasoning** in answer #3: 2 of 5 (slightly)
  Practise explaining why, not only what.
  > "trade-offs between speed and clarity"
- **Completeness** in answer #6: 2 of 5 (slightly)
  Practise covering each part of the question.
  > "trade-offs between speed and clarity"
- **Reasoning** in answer #6: 2 of 5 (slightly)
  Practise explaining why, not only what.
  > "trade-offs between speed and clarity"

Answer evidence

Answer #1 (Evaluated with AI)

- Relevance: 4 of 5
  > "trade-offs between speed and clarity"
- Completeness: 2 of 5
  > "trade-offs between speed and clarity"
- Reasoning: 2 of 5
  > "trade-offs between speed and clarity"
- Communication: 3 of 5
  > "trade-offs between speed and clarity"

You name the trade-off. Say which option you chose and why, with an example.

Answer #3 (Evaluated with AI)

- Relevance: 4 of 5
  > "trade-offs between speed and clarity"
- Completeness: 2 of 5
  > "trade-offs between speed and clarity"
- Reasoning: 2 of 5
  > "trade-offs between speed and clarity"
- Communication: 3 of 5
  > "trade-offs between speed and clarity"

You name the trade-off. Say which option you chose and why, with an example.

Answer #6 (Evaluated with AI)

- Relevance: 4 of 5
  > "trade-offs between speed and clarity"
- Completeness: 2 of 5
  > "trade-offs between speed and clarity"
- Reasoning: 2 of 5
  > "trade-offs between speed and clarity"
- Communication: 3 of 5
  > "trade-offs between speed and clarity"

You name the trade-off. Say which option you chose and why, with an example.

Communication observations

- Answer #1: 16 words; planned concepts mentioned: trade-offs
- Answer #3: 16 words; planned concepts mentioned: trade-offs
- Answer #6: 16 words; planned concepts mentioned: trade-offs

Speech metrics are not available: answers were given as text.

Topics to practise

- Programming Fundamentals (Programming Fundamentals)
  Completeness, Reasoning
- Project: Library Management System (Flask) (Project Understanding)
  Completeness, Reasoning

Recommended practice

- Practise covering each part of the question. (from answer #1, #3, #6)
- Practise explaining why, not only what. (from answer #1, #3, #6)

Retry

You can retry the topics to practise in a new interview.

Retry the topics to practise

Observed expression signals

These observations describe visible facial-expression patterns detected on your own device while you answered. They are experimental and approximate. They are not measurements of emotion, personality, confidence, honesty or suitability, and they are never used in any

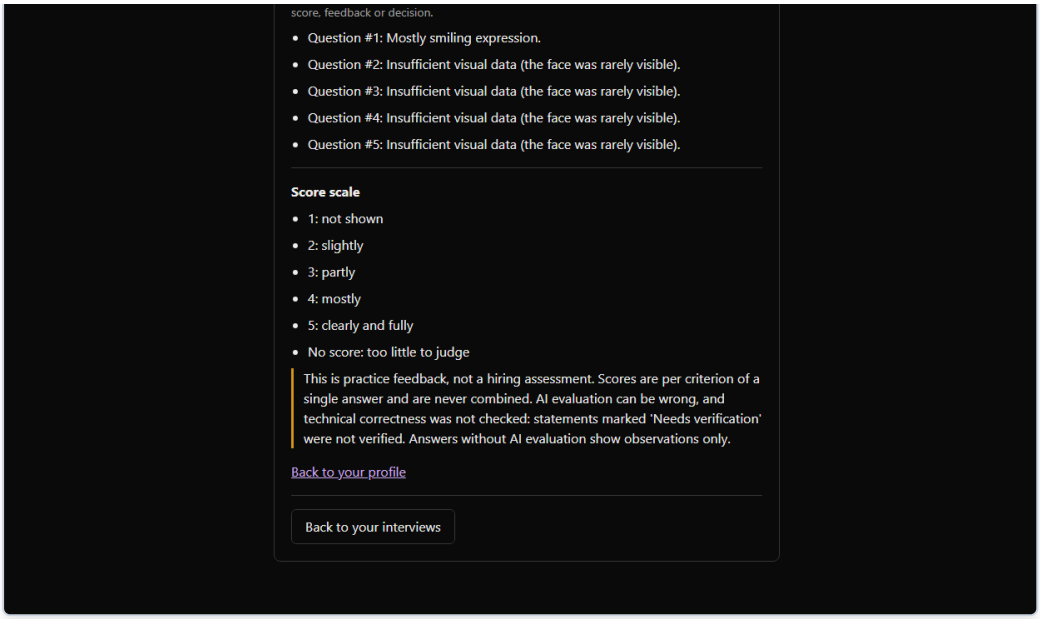


Figure B.6: Comprehensive Non-Aggregated Interview Evaluation Report

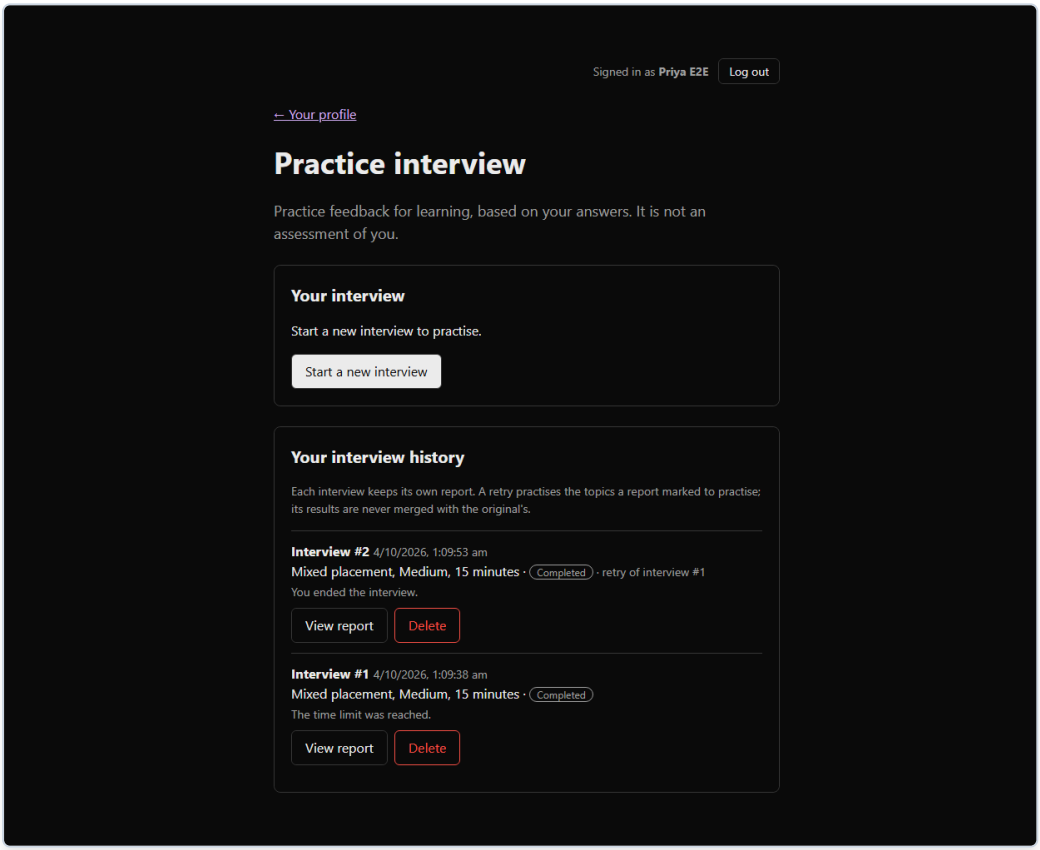
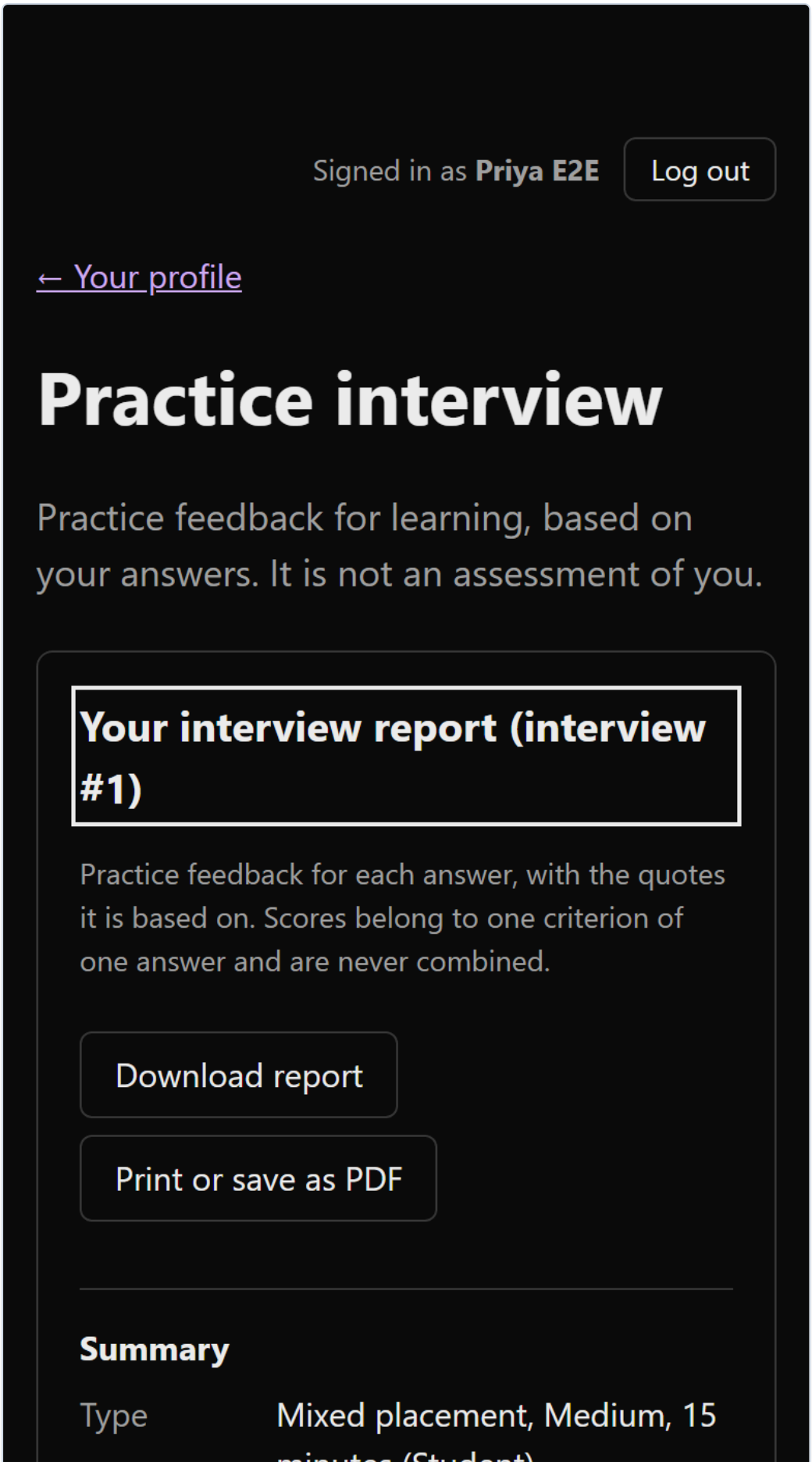


Figure B.7: Multi-Attempt Interview History and Targeted Retry Session Link


Signed in as Priya E2E
Log out
← Your profile
Practice interview
Practice feedback for learning, based on your answers. It is not an assessment of you.
Your interview report (interview #1)
Practice feedback for each answer, with the quotes it is based on. Scores belong to one criterion of one answer and are never combined.
Download report
Print or save as PDF
Summary
Type
Mixed placement, Medium, 15

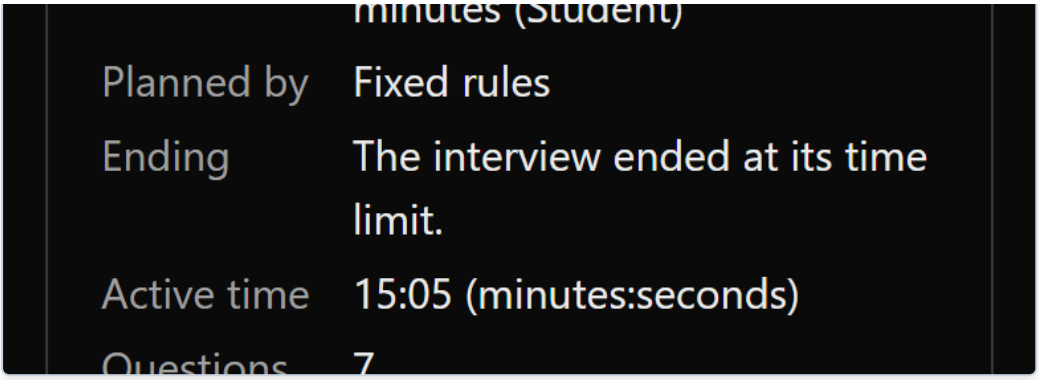


Figure B.8: Responsive Mobile View of Practice Report and Feedback

REFERENCES

BOOKS:

- Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley Professional.
- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design (3rd ed.). Prentice Hall.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

RESEARCH PAPERS:

- Achiam, J., et al. (2023). GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.
- Bommasani, R., et al. (2021). On the Opportunities and Risks of Foundation Models. arXiv preprint arXiv:2108.07258.
- Dwork, C., & Roth, A. (2014). The Algorithmic Foundations of Differential Privacy. Foundations and Trends in Theoretical Computer Science, 9(3–4), 211–407.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). 'Why Should I Trust You?': Explaining the Predictions of Any Classifier. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 1135–1144.
- Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS), 30, 5998–6008.

TECHNICAL STANDARDS & DOCUMENTATION:

- FastAPI Framework Documentation. (2026). https://fastapi.tiangolo.com/
- Next.js App Router Documentation. (2026). https://nextjs.org/docs
- PostgreSQL 16 Documentation. (2024). https://www.postgresql.org/docs/
- Pydantic V2 Documentation. (2024). https://docs.pydantic.dev/
- SQLAlchemy 2.0 Unified Documentation. (2024). https://docs.sqlalchemy.org/
- W3C Web Content Accessibility Guidelines (WCAG) 2.1. (2018). https://www.w3.org/TR/WCAG21/